

Serial Protocol Decode — Keithley DMM6500, DAQ6510, DMM7510

A UART decoder that runs **on** a Keithley bench instrument. It digitizes the line with the instrument's own digitizer, recovers the baud rate, frame format and idle polarity from the signal, and shows the bytes on the front panel as text or hex. No host, no logic analyser, one probe.

Ian Ameline · **version 1.05 — beta** · MIT licence (see [LICENSE](#))

docs/MANUAL.md	using it: hooking up, the screen, the buttons, what it copes with, where it fails
docs/REFERENCE.md	every measured number, and the firmware limits behind them
docs/BENCH.md	the harness: the seeded sweep, the release gate stage by stage, reproducing a failure

Ships as `Serial_Decode.tspa`, installed from a USB key through the instrument's own **Manage Apps** screen. Written in TSP — Lua **5.0.2** embedded in the firmware, so the sources avoid `#`, `%`, `string.gmatch` and bitwise operators throughout.

What to know before trusting it

Tested on a DMM6500 and nothing else. Every measured number in these documents came off that one instrument. See [Which instruments](#).

Flow control is verified electrically but never closed as a loop. One credit pulse per armed capture, measured on a scope, with nothing that waits for it ever on the other end. The width is not settable on this firmware, so a receiver needs an edge-triggered input rather than a polling loop. Figures and the gate-by-gate account of where it fails are under **Triggering, in both directions** in [REFERENCE.md](#).

Automatic rate detection has two known failures, both rare (together **under 0.5 %** of bench points) and both fixed by typing the rate in: a short pattern repeated over and over can measure at a small multiple of its true rate, and a logic low near **-2 V** can misdetect. Both are characterised with their numbers under **Known failures of automatic rate detection** in the [manual](#).

Nothing stops a long job early. A touch press cannot be delivered while a script runs, and the front-panel TRIGGER key does **not** deliver `trigger.EVENT_DISPLAY` during a panel-initiated run, so a recording runs to its stated size. That key does still *gate* an armed capture under `Options ▶ Trigger = Trigger key`.

One press is the whole interaction. A screenful is ~240 bytes; a recording holds 8 192 or 32 768 bytes to a file, decoded and filed without stepping; beyond that, flow control makes the window a chunk size and credits the device until it stops sending. Ceilings and arithmetic are in the [manual](#).

Endurance, measured — and reproduced

Two independent 17-hour soaks, each **6 laps of the full 1 683-point matrix, 10 098 capture-and-decode cycles on one power cycle**, with **no instrument event logged, no buffer or display object leaked and no error left behind**. They ran on different cabling, with a change to the app's state handling between them:

	duration	failures	drift
first	17.14 h	318	-0.9 %

	duration	failures	drift
second	17.17 h	313	−1.5 %

Within 0.2 % on duration and 1.6 % on failure count across 20 196 cycles, and **lap time fell in both runs** — the measurement that catches a leak, since a leak wedges the instrument without failing a point. **3.1 % of those points fail, against a matrix built to attack the decoder.** That failure set concentrates in four vectors, is mostly intermittent, and matches across the two runs more closely than one run matches itself. A third, 7-hour soak covers the recording path with zero failures. The numbers are under **Endurance** in [REFERENCE.md](#).

Which instruments

The app needs a digitizer reachable as `dmm.digitize`, the **touchscreen app API** (`display.create` and friends), and **Lua 5.0.2**. Every digitizer below runs at **1 MS/s**, and the **panel is pixel-identical across the whole TSP range**, so the app's screens carry over as they are.

DMM6500	Tested , firmware 1.7.17a. 16-bit digitizer — " <i>maximum resolution 16 bits</i> ", specifications, April 2018
DAQ6510	Should run unmodified, on the strongest grounds of the three: it shares the UI board and the acquisition board with the DMM6500, only the channel-board plugin differing. Untested
DMM7510	Should run unmodified: 18-bit digitizer, better acquisition boards. Untested
2461 SMU	Would need a port — dual 18-bit digitizers , reached as <code>smu.</code> rather than <code>dmm.</code>
2470 SMU	No. " <i>Digitized measurements are not a feature on the 2470</i> " — its reference manual, rev D, October 2024
2450 / 2460 SMU	Almost certainly not, by absence rather than denial: neither claims a digitizer, and the 2450 reference (rev D, May 2015) documents <code>smu.measure.*</code> with no digitize function

The timing figures transfer; the analog ones may not. Every rate figure in [REFERENCE](#) descends from the sample rate and the reading buffer's throughput. Resolution is the one axis this app is indifferent to, thresholding each sample into a one or a zero. What is measured *through the acquisition board* — the tolerance envelope, the level and offset limits, the −2 V band — is the part to re-measure: the DAQ6510 shares that board, so a deviation there is a real finding, while the DMM7510's differs and those figures may move either way. **A port is an acquisition question only** — the display layer carries over untouched, so the 2461's would be the 14 `dmm.*` symbols and nothing else. The panel constants are under **Firmware limits worth knowing** in [REFERENCE.md](#).

The `.tspa` header decides where it installs, independently of whether the code would run. The build declares `$Product: DMM6500, DAQ6510, DMM7510`. SMUs are absent: `$Product:` means "*can run the script without errors*", and an SMU would install and then fail at the first capture. `DMM7510` is not in the value set Keithley's app-header spec publishes, though Keithley ship TSP apps of their own declaring it — an install failure is the symptom if a firmware validates the field strictly, and this field is the first thing to revert.

If you run this on anything but a DMM6500, please say so — [open an issue](#). Every row above except the first is inference from API surfaces and vendor documents, and one report from real hardware outweighs all of it. What makes a report useful: `localnode.model` and `localnode.version`, whether **Manage Apps**

offered the app at all, whether both screens built, and whether a capture decoded. If it works, the figures worth re-measuring are the acquisition-board ones above rather than the rate ladder — those should already hold.

Before releasing it to anyone

Three gates, each about ten times the cost of the one before it. Run them in order.

```
python3 tools/release_sweep.py --offline      # ~1 min, no instruments
python3 tools/bench_smoke.py                  # 11 min, both instruments
python3 tools/soak.py --hours 17 --suites formats,plan --skip-vectors v95,v96
python3 tools/release_sweep.py                # the whole thing, instruments included
```

The smoke gate covers the whole rate range in 11 minutes. Four waveforms — the fox and a random payload, each in 8N1 and 7E1 — paired crosswise, so one pair takes the 22 standard baud rates and the other the 21 drawn ones, and each rate family faces both formats and both content classes. Then all 45 button presses. 86 cells in 9.5 min; presses in 1.9.

No version is tagged without at least 8 hours of soak, and the last two took 17. A soak reports a **failure rate per test point**; a sweep reports one pass or one failure. A lap is 1 683 cells and about 2.9 hours, so 8 hours is the least that separates "fails every lap" from "failed once", and 17 gives six laps.

Code changes do not get pushed without passing the smoke gate. On a pass, `bench_smoke.py` writes a receipt holding a hash of `tsp/` and `tools/`; a `pre-push` hook recomputes it and refuses if either tree has moved since.

```
ln -sf ../../tools/hooks/pre-push .git/hooks/pre-push
```

Documentation-only pushes are unaffected; neither tree is hashed. When the bench is unavailable — mid-soak, say — `SMOKE_OVERRIDE="reason" git push` proceeds and prints the reason.

Every lap draws its waveform order, its non-standard rates and the wait before each capture from the lap number, so `--iteration N` rebuilds any lap exactly without running the ones before it, and a failure replays offline in seconds.

docs/BENCH.md has the rest: every stage of the release gate and what it checks, the auditable record a full sweep leaves behind, what state the instruments have to be in before it will start, how to replay a failing cell offline, and the hazards worth knowing.

Layout

tsp/	the app. <code>serial_core</code> acquisition, <code>uart_decode</code> framing, <code>chunk_decode</code> resumable decode, <code>serial_ui</code> panel, <code>serial_app</code> orchestration
tools/	harnesses. <code>release_sweep.py</code> is the entry point; the rest are the authorities it calls
docs/	the manual, instrument references, panel mockups
notes/	bring-up log, findings, handoffs

`tsp/midi_decode.tsp` and `tsp/lin_decode.tsp` are complete and tested but **not shipped** in version 1 — the LIN checksum has never been checked against a real frame. Re-adding either is one line in `tools/package_tspa.py`; the app discovers what it has at runtime.

Bench

Three instruments over LAN: the DMM6500 under test, an SDG2122X generating the stimulus, an SDS1204X-E verifying the stimulus is what it claims to be. `tools/instruments.py` holds the addresses and the hazards — repeated large waveform uploads wedge the generator's LAN service, and calibration state is off limits on both.